

Executive Summary

Extending AJN's genre system requires a holistic approach covering API design, taxonomy standards, backend integration, search, UI, and governance. We propose a **controlled-vocabulary API** that lets producers and users tag items with canonical genre labels (e.g. "Comedy", "Science fiction") and synonym variants. These tags will be normalized against a curated ontology (possibly hierarchical) and mapped to common schemes (e.g. Schema.org's CreativeWork.genre, MusicBrainz genres, or IAB content categories) for consistency. Producers will emit `genres` in their `Program` objects, the registry will upsert them, and `libraryService` will project them onto `LibraryItem`s without altering the source identity chain. The search layer will normalize user queries (lowercasing, fuzzy matching, autocomplete) and match against the genre index. The UI will support tag autocomplete and a moderation queue for new genre suggestions, ensuring provenance (no fake IDs) and usability. We will add CI/test coverage for genre fields (unit tests for normalization, regression tests to preserve identity invariants). Migration of existing items involves adding a `genres` array field, seeding it from current categories/tags, and updating storage (e.g. ALTER TABLE or JSON migrations). Finally, we enforce rate limits and moderation to prevent abuse, just as with other user-supplied data.

Key points: Schema.org already uses `genre` (e.g. `itemprop="genre">Science fiction</span>【32†L121-L129】`), and Internet Archive items use "Topics" (tags like "fiction", "American novel")【16†L263-L270】. We will align our system with these ideas. A robust taxonomy helps users find content via search or guided filters【50†L245-L253】. The following report details (1) API and data design, (2) taxonomy strategy, (3) producer/registry integration, (4) search ingestion, (5) UI/UX, (6) CI tests, (7) migration plan, (8) security/abuse mitigation, and (9) sample code.

1. Genre API Design

- **Endpoint Design:** We add REST endpoints to manage genres. For example:
 - `GET /genres` – list all allowed genres (with optional `?q=` fuzzy search).
 - `POST /genres` – create a new genre (admin/moderator only) with validation.
 - `GET /items/{id}` – existing; now returns `genres: string[]` in the JSON.
 - `PATCH /items/{id}/genres` – update genres on an item (with body `{"genres": [...]}`).
 - `GET /search?genre=Comedy` – filter search results by genre tag.
- **Payload and Validation:** Genre names are Unicode strings (max length e.g. 64), normalized to lowercase and trimmed. Validation rules include: no empty or placeholder (e.g. "BigBuckBunny"), no disallowed characters (only letters, spaces, and basic punctuation). New genres must not duplicate existing ones modulo synonyms. E.g. "SciFi" would normalize to "science fiction". If a user tries to add a genre, it is checked against the controlled list or queued for review. API returns HTTP 400 on invalid names or disallowed source types.
- **Approval Workflow:** Only verified producers (approved `sourceClass`, e.g. `archive_org`) can programmatically tag items. User-added genres go into a moderation workflow (see section 5). This

ensures provenance: **no synthetic identities or unapproved classes** introduce tags. All tags must ultimately originate from a trusted source or be vetted.

2. Taxonomy Normalization & Ontology

- **Controlled Vocabulary:** We curate a canonical genre list (seeded from common categories like “Documentary”, “Comedy”, “News”, etc.), stored as a table or config. Each entry may have synonyms (e.g. “Sci-Fi”, “SF” → “Science fiction”). We normalize on insert: convert to lowercase, remove diacritics, map synonyms. This controlled set avoids near-duplicates.
- **Synonyms and Mappings:** We maintain a synonym mapping file or table. For example:
 - key: `science fiction`, synonyms: `sci-fi, sci fi, SF, science-fiction`.
 - key: `children`, synonyms: `kids, kids'`, etc. On input, we canonicalize to the key. This ensures that search for “Sci-Fi” matches “Science fiction” in data.
- **Hierarchical Taxonomy:** We propose a *two-tier hierarchy*. Top-level genres (e.g. **News, Science, Arts, Culture**) each have sub-genres (e.g. under **Arts**: *Film, Music, Documentary*). A simple example:

```
News
- Politics
- Business
Science
- Astronomy
- Biology
Film
- Documentary
- Drama
- Comedy
```

This helps in UI filters and browsing. It mirrors common library classifications (e.g. DDC 700s for art, 800s for literature) and also aligns with IAB’s Content Taxonomy (which is hierarchical [34†L65-L73]). We store parent-child links so we can filter at any level.

- **Crosswalks to Standards:** Wherever possible, we tag genres with external IDs:
- **Schema.org:** We use the `genre` property on CreativeWork types [32†L121-L129] . For example, `"Science fiction"` in our API corresponds to `itemprop="genre"`.
- **IAB Content Taxonomy:** If aligning to advertising context, our top categories map to IAB 3.0 codes (e.g. “News”→IAB1, “Movies”→IAB22). The IAB site notes its taxonomy is a “common language” for describing content [34†L28-L36] .
- **Library Classifications (DDC/LOC):** Broadly, we can note DDC classes (e.g. 700s = Arts) for internal grouping, but won’t fully adopt DDC numbers except possibly as optional metadata.

- **MusicBrainz/others:** For audio content, we may allow MusicBrainz tags (e.g. “Jazz”, “Classical”), storing them as genres, possibly linking to MBIDs if needed.
- **Language and Locale:** We normalize language (e.g. “horror” vs “ホラー”). If supporting multilingual labels, we store canonical English and optionally localized display names. Search can detect language and map to the correct genre.

Table: Taxonomy Source Comparison

Schema / Source	Coverage	Structure	Notes
Schema.org (genre)	Free-text labels	None (flat)	Used on CreativeWork [32†L121-L129] ; no fixed list, text values
IAB Content Taxonomy	~1500+ categories	Hierarchical	Broad online content taxonomy [34†L28-L36] ; includes “genre” facet
LC/LOC Classifications	Bibliographic subjects	Hierarchical	Broad library subject headings; fine-grained but not user-friendly labels
Dewey Decimal (DDC)	Topical classes (000–999)	Hierarchical	Library-focused disciplines (e.g. 700s = Arts)
MusicBrainz genres	Music-specific	Flat or Folksonomy	Community tags for recordings; large but not authoritative
AJN Controlled List (proposed)	Multimedia content types (film, news, science, etc.) + subgenres	Hierarchical	Custom list combining IAB/ Schema.org categories, vetted by editors

[\[32†L121-L129\]](#) [\[34†L28-L36\]](#)

3. Producer & Registry Integration

- **Program Model Extension:** Update the `Program` data model to include a `genres: string[]` field (or `genreIds`). For example:

```
interface Program {
  id: string;
  channelId: string;
  guideId: string;
  sourceClass: string;
  sourceId: string;
  assetId: string;
```

```

title: string;
mediaUrl: string;
mediaType: 'video'|'audio';
metadata: any;
genres?: string[];    // <-- new field for genres
}

```

Producers (e.g. TV news, Nova, Movies) should populate `genres` when emitting programs. For example, the Nova producer could tag all Nova episodes as `["Documentary", "Science"]`. Producers may derive genres from the source (e.g. "science-documentaries" guideId implies Science genre) or from manifest metadata.

- **Registry Ingestion:** In `guideRegistry.initializeCanonicalRegistry()`, when upserting `Program`s, include the `genres` array. The registry does not validate genre values (other than basic syntax) – it trusts producers' tags (all producers use approved sourceClasses like `archive_org`).
- **Library Projection:** In `libraryService.getCuratedLibraryProjection()`, modify mapping to include genres. For example:

```

export function getCuratedLibraryProjection(): LibraryItem[] {
  return getCanonicalPrograms()
    .filter(/* existing filters */)
    .map(program => ({
      ...,
      tags: program.metadata?.tags || [],
      genres: program.genres || [],    // preserve genres
      ...
    }));
}

```

Provenance check: Do not allow genre assignment during projection. i.e. *do not synthesize genres*. Only use `Program.genres` as given.

- **Validation:** As part of integration, add checks that any genre strings match the normalized controlled vocabulary. For instance, in `upsertCanonicalProgram`, one could normalize all incoming genres or reject unknown terms. This prevents rogue tags. E.g. `if (!isAllowedGenre(g)) throw Error("Unknown genre");`. (Alternatively queue new terms for curation.)
- **Flow:** Producer → guideRegistry → getCanonicalPrograms() → libraryService → LibraryItem. (See flowchart below.)

```

flowchart LR
    Producer["Producers\n(e.g. TV, Nova, Movies)"] -->|emit Program with genres| Registry["guideRegistry\n(canonical Program list)"];
    Registry -->|gives list of Programs| Library["libraryService\n(curated LibraryItems)"];
    Library -->|indexes for search| Search["Search Engine"];
    Query["User query"] -->|normalize (lowercase, synonyms, etc.)| Norm["Normalization/Mapping"];
    Norm --> Search;

```

4. Search & Query Processing

- **Query Normalization:** User queries may include genre keywords (e.g. `genre:comedy`, or just “comedy”). Before matching, we normalize: lowercase, strip punctuation, map synonyms. E.g. query `"Sci-Fi & fantasy"` → normalize to `"science fiction", "fantasy"`. Implement this in the search service or API layer.
- **Autocomplete & Suggestion:** Provide autosuggest on genre terms. When user types in a search box, run their prefix against the genre list (`GET /genres?q=<prefix>`). This returns possible completions (like “thriller”, “thriller movie”). Use fuzzy matching (e.g. Levenshtein) so “scfi” suggests “sci-fi”.
- **Language Detection:** If content is multilingual, detect query language. For example, a query in Chinese for a known genre (“科幻” meaning “Sci-Fi”) could be mapped to the English genre key “science fiction”. Use a small translation map for genre words, or rely on search engine’s language tools.
- **Search Hooks:** When indexing `LibraryItem`s, index the `genres` array. The search engine (e.g. Elasticsearch or in-memory search) should treat genres as a special facet. We can allow faceted search by genre. The API can support `?genre=Comedy` to filter results where `"Comedy" ∈ item.genres`.
- **Synonym Expansion:** Optionally, maintain a synonym list for search so that searching “drama” matches “theatrical drama” or subgenres.
- **Provenance Invariance:** Ensure search indices do not mix provenance. Only index items with approved sourceClass (the existing libraryService filter already ensures that). The genre system does not alter identity fields.

5. UI/UX & Moderation

- **Genre Tagging UI:** In item-edit or upload forms, provide a “Genres” field with autocomplete dropdown from allowed list. Users can select existing genres or start typing to get suggestions. If a user types a new term (not in list), show an “Add new genre” option (if allowed for the user’s role).

- **User-Added Genres:** We assume only *trusted users* can add new genres. E.g. logged-in editors or curators. When a new genre is proposed, it enters a **moderation queue**. An admin reviews the suggestion, possibly renaming it to an existing synonym or adding it to the controlled vocabulary. This prevents spam or trivial duplicates.
- **Display:** On the Library or Program detail pages, display genres as tags or pills (e.g. “Genres: Documentary, Science”). Allow filtering by clicking a genre tag.
- **Error Handling:** If a user tries to add a disallowed or duplicate genre, show a validation error. For example, adding a category already covered (e.g. “Film” if it’s already under a broader tag) should be flagged or auto-merged.
- **Consistency:** The UI should *never* allow editing source identity (sourceClass, sourceId, assetId). Only genres and metadata. Any attempt to manipulate sourceIds (e.g. in JSON) should be blocked by server logic.

6. CI/Test Plan

- **Unit Tests:**
 - *Normalization:* Test that varied inputs map to canonical genres. E.g. `"SciFi" → "science fiction"`, `"romcom" → "romance"`.
 - *API:* Test the new endpoints return the expected schema and enforce validation (e.g. POST /genres with invalid chars returns 400).
 - *Integration:* For a mock `Program`, verify that after `upsertCanonicalProgram` and `getCuratedLibraryProjection`, the `LibraryItem` contains the same `genres` array and that no extra or synthetic genres appear.
- **Regression Tests:** Update existing suite (e.g. `test-library-integrity.js`) to include checks on genres:
 - Assert that every `LibraryItem.genres` is non-empty only if the program’s `genres` was set (no new blank tags).
 - Reject any “synthetic” genre like “genre-1234” or placeholders (similar to the BigBuckBunny test).
 - Ensure no “m3u_live” or unapproved source sneaks in (existing tests cover sourceClass).
 - Possibly add a test that `getScheduleForGuide` or similar includes `genres` in its output.
- **CI Integration:** Include the new tests (e.g. `npm run test:genres`) in the pipeline. Re-run full suite on PR to catch any breakage in identity rules. For example, confirm that adding `genres` doesn’t cause any test to fail on whitespace (the existing tests check `program.sourceId` and `program.assetId` still have no synthetic patterns).

7. Migration Strategy

Object	Current Fields	Proposed Change
Program	id, channelId, guideId, sourceId, assetId, title, mediaUrl, metadata.tags	+ genres: string[] (nullable)
LibraryItem	id, title, description, category, tags, year, format, guideId, channelId, sourceId, assetId	+ genres: string[]

- **Database Changes:**

- If SQL: `ALTER TABLE Programs ADD COLUMN genres TEXT[];` (or JSON). `ALTER TABLE LibraryItems ADD COLUMN genres TEXT[];`
- If NoSQL/JSON: Just include a new field in the stored document.
- **Backfill:** For existing items, we may map current `category` or `tags` to genres. E.g. if `category="news"`, set `genres=["news"]`. Otherwise, start with empty arrays and allow editors to tag.
- **Example SQL (PostgreSQL):**

```
ALTER TABLE Program ADD COLUMN genres text[];
UPDATE Program
  SET genres = ARRAY[LOWER(category)]
  WHERE category IS NOT NULL;
ALTER TABLE LibraryItem ADD COLUMN genres text[];
UPDATE LibraryItem
  SET genres = ARRAY[LOWER(category)]
  WHERE category IS NOT NULL;
```

- **Migration Table:** | Table | Before | After | |-----|-----|-----| |
Program | No `genres` field. | Add `genres: string[]`. Map `tags` → `genres` if relevant. | |
LibraryItem | No `genres`. | Add `genres: string[]`; copy from *Program*. |

- **Repository Changes:** Update TS interfaces (`Program`, `LibraryItem`) to include `genres`. Update any DB schema definitions or ORMs accordingly.

Migrating existing data will ensure no disruptions. New code paths default to empty list if `genres` is missing.

8. Security, Privacy & Abuse Mitigation

- **Rate Limiting:** Apply rate limits on genre-related API endpoints to prevent spam or brute-force. For example, limit `/genres` POST to 5 requests/minute per IP or user.
- **Authentication & Authorization:** Only authenticated, authorized users (e.g. curators) can add new genres or update item genres. Public search queries by genre are read-only.

- **Input Sanitization:** Since genres are user input, sanitize to avoid injection (though plain strings, disallow HTML/JS). Reject overly long or weird entries.
- **Provenance Checks:** Maintain the existing rule: any item in the curated library must have a valid `sourceClass` (e.g. `archive_org`). We never allow genres to override or hide the source identity. The new workflow must not introduce “synthetic” identity patterns (see CI tests).
- **Moderation:** New or rare genre proposals go into a queue. Moderators verify they aren’t obscene, politically charged, or trademarked. (For example, disallow “Hitler documentary” as a genre.) Use a simple review UI or Slack/email alerts to admin.
- **Privacy:** Genre tags are about content, not people, so privacy risk is low. However, if user profiles suggest genres, ensure anonymity. Rate-limit or CAPTCHA on genre suggestions to prevent bots.
- **Abuse Example:** A malicious user might try to add a genre like “free money”. Moderation will catch this since it doesn’t fit our controlled list. The system should automatically reject blatantly spammy tags.

9. Sample Code & Tests

TypeScript Interfaces:

```
// Add genres to existing types
interface Program {
  id: string;
  channelId: string;
  guideId: string;
  sourceClass: 'archive_org' | 'ajr_rss' | 'ajr_archive';
  sourceId: string;
  assetId: string;
  title: string;
  mediaUrl: string;
  mediaType: 'video' | 'audio';
  metadata: { description?: string; year?: number; tags?: string[]; };
  genres?: string[]; // new field
}
interface LibraryItem {
  id: string;
  channelId: string;
  guideId: string;
  title: string;
  description: string;
  category: string;
  tags: string[];
  year: number;
  format: 'video' | 'audio';
```

```

sourceId: string;
programId: string;
assetId: string;
archivePath: string;
sourceClass: Program['sourceClass'];
isCurated: boolean;
genres: string[];      // new field (always present)
}

```

Producer Example: (setting genre in a canonical program)

```

// src/services/producers/novaProducer.ts (excerpt)
import { normalizeGenre } from '../../utils/genreUtils';

export function getNovaCanonicalPrograms(): Program[] {
  const sourceId = deriveSourceId('archive_org', NOVA_ITEM_ID);
  const baseGenres = ['Documentary', 'Science'];
  return manifestEpisodes.map(item => ({
    // existing fields ...
    sourceId,
    assetId,
    title: `NOVA Wonders: ${item.title}`,
    // ...
    metadata: { description: ..., year: 2018, tags: [...] },
    genres: baseGenres.map(normalizeGenre), // attach genres
  }));
}

```

Here, `normalizeGenre` would lowercase and map synonyms. For example `"Science"` → `"science"`.

LibraryService Update: (preserve genres)

```

export function getCuratedLibraryProjection(): LibraryItem[] {
  const programs = getCanonicalPrograms();
  return programs
    .filter(program => /* existing filters */ true)
    .map(program => ({
      id: program.id,
      title: program.title,
      /* ... */
      tags: program.metadata?.tags || [],
      genres: program.genres || [],      // include genres
      sourceId: program.sourceId,
      assetId: program.assetId,
      sourceClass: program.sourceClass,
    }));
}

```

```

        isCurated: true,
    }));
}

```

Unit Test Snippet: (using `node:assert/strict`)

```

// test/moviesClassicsProducer.test.js
import assert from 'assert';
import { buildMoviesClassicsPrograms } from '../src/services/producers/
moviesClassicsProducer';

describe('Movies Classics Producer', () => {
  it('expands multi-file items with shared sourceId', () => {
    const mockManifest = [{
      identifier: 'antology-pack',
      title: 'Classic Pack',
      year: 1970,
      files: [{ name: 'M1.mp4' }, { name: 'M2.mp4' }]
    }];
    const programs = buildMoviesClassicsPrograms(mockManifest);
    assert.equal(programs.length, 2);
    assert.equal(programs[0].sourceId, programs[1].sourceId);
    assert.notEqual(programs[0].assetId, programs[1].assetId);
  });
});

```

```

// test/genreNormalization.test.js
import { normalizeGenre } from '../src/utils/genreUtils';
describe('Genre Normalization', () => {
  it('maps synonyms to canonical form', () => {
    assert.equal(normalizeGenre('Sci-Fi'), 'science fiction');
    assert.equal(normalizeGenre('Sci fi'), 'science fiction');
  });
  it('lowercases and trims', () => {
    assert.equal(normalizeGenre(' Comedy '), 'comedy');
  });
});

```

【32†L121-L129】 【16†L263-L270】

Each code example shows how genres enter the pipeline and are preserved. Regression tests (similar to the above) should ensure that no identities are lost or faked when adding this feature.